AGENT TRACE ANALYSIS RESEARCH GUIDE

Trace Observability & Diagnostic Systems Synthesis — Comprehensive guide on agent trace architectures, empirical benchmarks, and observability solutions.

TECHNICAL DOCUMENTATION: HOW TRACES LOOK LIKE

Local software engineering agents operate directly on developer workstations, writing machine-readable, append-only records of conversational turns and sub-actions to local disk structures rather than centralized remote databases [cite: 3, 4, 6].

Storage Directories & File Serialization

- **Path Mapping:** In macOS and Linux environments, projects are isolated under `~/.claude/projects/<url-encoded-project-path>/`, while Windows systems route to `%USERPROFILE%\.claude\projects\` [cite: 8, 9, 10, 17, 18, 19].
- **Append-only Logs:** Individual sessions are logged as JSON Lines (JSONL) files inside a `sessions/` subdirectory, identified by a unique session UUID [cite: 12, 20]. Logs append a single JSON record per event to ensure crash resilience during long-running tasks [cite: 20].
- **Envelope Schema:** Records share a consistent structure [cite: 21]:

```
Envelope {type, uuid, parentUuid, timestamp, sessionId, cwd, message}
```

- **DAG Representation:** Incorporating the `parentUuid` allows sessions to be modeled as a Directed Acyclic Graph (DAG) rather than a sequential list [cite: 23]. This tracks execution branching, retries, edits, and orchestrator-to-subagent coordination [cite: 23].
- **Alternative Architectures:** Open-source engines like OpenClaw utilize JSON5 configurations alongside markdown workspaces, writing short-term actions to `memory/YYYY-MM-DD.md` and long-term context to a root `MEMORY.md` index [cite: 24, 25].

Standardized Telemetry: OpenTelemetry GenAI Conventions

To prevent lock-in, agents adopt OpenTelemetry (OTel) GenAI semantic conventions, representing execution as a tree of hierarchical spans [cite: 162, 164, 167]. The root span is categorized as `invoke_agent`, below which the trace interleaves model interaction spans (`chat`) and tool execution spans (`execute_tool`) [cite: 168, 169]. To preserve privacy while debugging, raw content payloads are isolated into span events (`gen_ai.content.prompt` and `gen_ai.content.completion`), which can be redacted to comply with data governance frameworks [cite: 183, 184].

PAPERS: AGENT TRACES ANALYSIS

Type 1: Traces on Benchmarks

Audits of execution trajectories from benchmarks such as SWE-bench, GAIA, and SWE-chat reveal various cognitive and operational failure modes [cite: 57].

- **Cognitive Lock-In & Fixation:** During deep repository exploration, analytical thoroughness can cause models to lock in on incorrect bug interpretations [cite: 59, 60]. Once committed to a flawed premise, models dedicate subsequent tool calls to validating the mistake, ignoring contradictions in terminal or test outputs (the "consistent wrong" phenomenon) [cite: 61, 63]. Standard self-reflection prompts often reinforce this mistake rather than correcting it [cite: 64].
- **Infinite Loops & Cost Cycles:** Operating in ReAct loops without explicit planning or termination constraints leads to repetitive tool executions [cite: 82, 83, 84]. This is accelerated by context window pollution, where a growing trajectory degrades the signal-to-noise ratio, causing agents to ignore termination directives and loop endlessly [cite: 86, 87].
- **Advanced Auditing Frameworks:** Frameworks decompose long traces to isolate the earliest decisive error step (the initial action introducing a breakdown) [cite: 109, 110]:
    - **TrajAudit:** Uses a multi-stage investigator agent to filter verbose logs via pattern matching and applies failed test reports to identify suspicious execution windows [cite: 112, 113, 114, 115].
    - **Reflect:** Employs intervention-supported error attribution via execution grounding (validation via running code), prefix-preserving replay (freezing state up to the suspected step), targeted intervention (injecting corrections), and controlled replay (replaying forward) [cite: 273, 117, 118, 119, 120, 121, 122].
    - **DRIFT:** Uses claim-centric auditing, translating logs into semantic spans and mapping the propagation of false claims [cite: 123, 124, 125].
    - **HarnessFix:** Employs trace abstraction (HTIR) to explicitly link temporal order, input provenance, and control flow [cite: 127, 128, 129, 130].

Type 2: Human in the Loop (Popular Coding Agents – Claude Code)

Analysis of real-world developer interactions via systems like Claude Code, Codex, or Gemini CLI (using datasets like SWE-chat) exposes significant gaps between autonomous generation and successful integration [cite: 66].

- **Bimodal Modality:** Sessions are highly bimodal: 41% consist of agent-dominated execution ("Vibe Coding"), while 23% involve human-dominated manual authorship [cite: 67, 68].
- **Structural Churn:** Agent execution remains highly inefficient; only 44% of agent-generated code survives into final user commits, pointing to heavy structural churn, redundant modifications, and an increased risk of security vulnerabilities [cite: 69, 70].
- **Developer Intervention:** Suboptimal generations lead to human intervention, with users pushing back via corrections, failure reports, or interruptions in 44% of conversational turns [cite: 71, 72, 73, 74].
- **Multi-Turn State Blindness:** While turn-local superficial errors (like syntax) are caught by simple single-turn evaluators, cross-turn state failures (confirm-gate violations, context shifting, missed transitions) are consistently missed by monolithic LLM judges [cite: 75, 76, 77, 79]. These multi-turn defects are codified in the TRAIL error taxonomy, which classifies over 20 distinct span-level error categories [cite: 80].

DATASETS: HOW TRACES LOOK LIKE

Trajectory datasets represent chronological collections of agent thought paths, environment modifications, terminal actions, and tool calls, acting as foundations for training models and validating diagnostic tools [cite: 4, 188].

- **SWE-chat:** Features 5,851 real-world developer sessions and over 355k tool calls [cite: 190]. It captures transcripts, model thoughts, tool calls, and git diffs, providing line-level attribution of agent vs. human authorship in the wild [cite: 194, 195].
- **SWE-Lego:** Contains 32k tasks and 18k validated expert trajectories [cite: 196]. SFT training pipelines use step-level error masking to exclude tokens associated with tool failures and failing tests, ensuring the model learns only valid problem-solving patterns [cite: 197].
- **RootSE:** A failure benchmark with 93 repository-level instances and 4,500+ steps [cite: 190]. It provides a testing ground for isolating the earliest decisive error amidst verbose code noise [cite: 201, 202].
- **TELBench:** A span-level error localization benchmark with 1,000 instances (2,790 expert-annotated trajectories) designed to evaluate trace diagnostic models amidst exploration noise and logical drift [cite: 190, 203, 204].
- **Additional Benchmarks:** SWE-bench Verified evaluates repository-level code repair, Terminal-Bench 2.0 tracks command-line execution and environment drift, GAIA focuses on open-web deep-research synthesis, and AppWorld assesses multi-app API automation [cite: 190].

COMPETITORS: AGENT TRACES ANALYSIS

Observability tooling splits between enterprise production tracing platforms and local, developer-centric auditing utilities [cite: 134].

Type 1: Analysis of AI Workflows in AI-Apps (LangSmith, LangFuse, ...)

Enterprise platforms focus on scalable telemetry ingestion, standardizing metrics and evaluations across concurrent cloud deployments [cite: 143].

- **Langfuse:** An open-source, production-focused tracing and prompt management platform [cite: 145]. It uses asynchronous batch exporting to collect traces without adding application latency and excels at cost tracking and prompt versioning, though it requires custom evaluator scripts [cite: 145, 146, 147].
- **Arize Phoenix:** An open-source, evaluation-first toolkit integrated with OTel [cite: 148]. It provides pre-built "LLM-as-a-judge" evaluators for relevance, hallucination, and toxicity, along with embedding drift analysis [cite: 149, 150].
- **LangSmith Engine:** A platform managing the agent improvement loop end-to-end [cite: 151]. Its specialized engine analyzes production logs, clusters failure paths, proposes test cases, and runs automated trace testing and CI/CD regressions [cite: 151].

Type 2: Analysis of Locally Saved Sessions of Popular Coding Agents

For local desktop interactions, enterprise SaaS tools can introduce privacy risks by exporting proprietary source code or may be overly complex [cite: 153]. Developers instead use local command-line utilities that parse disk JSONL traces [cite: 154].

- **agenttrace (luoyuctl):** A local-first CLI/TUI designed to audit execution logs from Claude Code, Codex, Gemini CLI, Aider, and Cursor [cite: 155]. It generates offline HTML reports, interactive TUI dashboards, and CLI health gates to analyze token spend, latency, and repetitive loops locally [cite: 156, 157].
- **cclogviewer (lm-assist):** A local web interface designed specifically to parse Claude Code projects from `~/.claude/projects/` [cite: 158]. It calculates API costs using real-time pricing models and applies static heuristics to detect repetitive, wasteful file reads [cite: 158, 159].
- **cchistory:** A lightweight utility extracting shell commands executed by agents within local session histories, allowing developers to search, filter, and reuse CLI interactions [cite: 159, 160].